Exam for the course DAT171 Object oriented programming in Python

Time: 11th June 2019 8:30-13:30

Teacher: Thomas Svedberg (031-772 1522), Mikael Öhman (073-6837674)

Permitted aids: Cay Horstmann: Python for everyone. Manuals and lecture notes are available on the computers.

Teacher will visit the rooms: Around 10:00 and 12:00

Formalities: In each file you hand in, you should write your examination code. Also enter the computer number (from the exam cover).

The documentation and lectures notes are available on `C:\__EXAM__`. Verify that these files are there immediately. Handing in the code should be done under the same folder: `C:\__EXAM__\Assignments\`, when you are finished. If you do not store the files here, they are not included in the exam. Save the files for each question in the appropriate folder, e.g. `C:\__EXAM__\Assignments\Question1\`. *Make sure you only hand in one solution for each question, otherwise you will receive **zero** points.* Complex sections of your code should have a descriptive comment of what is achieved.

When you finish the exam you should log out and fill in the (empty) exam cover page like normal and hand this in at the end of the exam.

Corrections: The results will be announced through Ladok, on or before June 26th. The review will be the same day (June 26th) 10:30-11:00 at Hörsalsvägen 7b, 3:rd floor.

Grading: There is a total of 25 points which yields grades at the standard 40%, 60%, and 80% limits:

```python
def grade(points):
    if points >= 20:
        return 'Grade 5'
    elif points >= 15:
        return 'Grade 4'
    elif points >= 10:
        return 'Grade 3'
    else:
        return 'Fail'
```

Question 1 (8p total)

Search engines use different methods for ranking the best pages. One such technique is to count how many other pages link to it. To do this, they scan through HTML pages, looking for all the links.

In HTML, links are specified as

```
Page with a link to <a href="www.google.com">google</a>.
Another link to <a href="http://www.amazon.co.uk/gp/deals">amazon deals</a>
HTTPS is also allowed (secure connection) like <a href="https://somebank.com/login">this</a>
```

Part A (3p)

Write a support function that extracts all links from a given text. It should work precisely like this example:

```
>>> x = extract_links('Links to <a href="http://www.amazon.co.uk/gp/deals">amazon deals</a>
and <a href="www.google.com">google</a>')
>>> print(x)
['http://www.amazon.co.uk/gp/deals', 'www.google.com']
```

You may assume that the links are correctly formatted. You may also assume that the links themselves do not contain the " character.

Hint: ***help(str)***

Part B (2p)

Write another support function that picks out the domain name from a link by stripping off everything but the domains name itself, like in these examples:

```
>>> a = extract_domain('https://www.amazon.co.uk')
>>> b = extract_domain('http://www.foo.org/stuff/index.html')
>>> c = extract_domain('google.se/search?q=kitten')
>>> print(a)
www.amazon.co.uk
>>> print(b)
www.foo.org
>>> print(c)
google.se
```

Hint: Domain names can not contain special characters, like **/**

Part C (3p)

Read through the file `links.html` and extract all the domain names by making use of the functions you wrote for parts A and B. Count the frequencies of links to each domain (e.g. 34 links to `'www.google.com'`, 10 links to `'www.chalmers.se'`, 5 links to `'dx.doi.org'`) Print the domain names, and the number of links to this domain to the screen in a nice tabular format, for example, like this:

```
| Domain           | Links |
+------------------+-------+
| www.google.com   |    34 |
| www.chalmers.se  |    10 |
| dx.doi.org       |     5 |
```

and so on.

You may assume that there are no line-breaks inside the `<a href...</a>` links.

Hint: `Counter` from the `collections` module

Hint: str.format allows for alignment. You start by specifying colon + direction + width, like `':>5'` for right align width 5. Try out `'{:>5}'.format(123)` in the python command line. You may also find multiplication of strings a useful feature; `'+'*10`

Do not submit multiple copies of Part A or Part B. You may write all of these in the same file. Handing in multiple solutions results in ZERO POINTS.

Question 2 (5p total)

In this task NumPy / SciPy **must** be used for any array-type of numerical data.

The NumPy / SciPy libraries consists of routines for efficient numerical calculations. Key to achieving good performance using these libraries is efficiently using the built in methods for slicing, selection and filtering of arrays, as well as avoiding explicit looping.

NOTE! You should use no loops whatsoever for this question!

Part A (1p)

- Read the data from the file named "values.txt" into NumPy-array `a`.
- Reformat `a` into array `b` that should have 2 blocks each of 4 rows and 3 columns. We define blocks as associated with the *first* index, rows the second index, and columns the third index; `[block, row, column]`
- Print `a` and `b` to the screen, include labels (i.e. `a = [[ ....`)

Part B (1p)

Slicing. Using `b` from above, separately print:

- block 0
- row 2
- column 1
- block 1 with the rows reversed

(i.e. 4 print statements)

Part C (1p)

Selection and filtering. Using `b` from above; Create mask-arrays:

- `c`: Marking all elements `< 50`
- `d`: Marking all elements `>= 50 and < 75`
- `e`: Marking all elements `>= 75`

Print the arrays above, with labels as in task A.

Use the mask-arrays above and print the elements from `b`:

- `< 50`
- `>= 50 and < 75`
- `>= 75`

Part D (2p)

For each block in `b`, we want to find and print the row with the highest sum of its values. To do this, you should perform the following sub-tasks:

For each block in `b`, find the row with the highest sum. Print the resulting row-sums.

Find the position of the highest value among the row-sums above (for each block).

Hint: Look at ***`argmax`*** *in NumPy*

For block 0 and block 1, respectively, print the row with the highest sum. Remember to print labels for all your output.

Remember: Any explicit loops in your code will render zero points for that task!

Question 3 (5p total)

In this task you **should not** use any **built-in** complex number functionality!

A complex numbers library

A complex number (written in Cartesian form) is a number $z = a + bi$ where a is the real part, b is the imaginary part and $i^2 = -1$. Binary operations are defined by e.g.

$$(a + bi) + (c + di) = (a + c) + (b + d)i$$
$$(a + bi) * (c + di) = (a * c - b * d) + (a * d + b * c)i$$

The absolute value is defined as $|a+bi| = \sqrt{a^2 + b^2}$. A complex number can also be written on polar form as $z = (r, \theta)$ where $r = |z|$ and $\theta = \arctan(b/a)$ (in the correct quadrant). Converting from polar form to Cartesian form is done with $a = r\cos(\theta)$ and $b = r\sin(\theta)$.

The complex module

In this task you will create a module suitable for doing some simple operations with complex numbers. The name of the module should be `complex`. It should contain the following:

- A `class` named `Complex` which is used to create complex numbers. It should contain:
  - Constructor such that `Complex(a, b)` creates the complex number `a + bi` and `Complex(a)` should create the complex number `a + 0im`.
  - Pretty string conversion: Numbers should be printed as:
    - `print(Complex(2, 3))` — `2 + 3im`
    - `print(Complex(2))` — `2 + 0im`
    - `print(Complex(2, -3))` — `2 - 3im`
  - Binary operations between `Complex` numbers. You only have to implement `+` and `*`. These functions should also work if one of the numbers is a real number as in `5 + Complex(3, 2)`.
  - Methods `real` and `imag` returning the real and imaginary component, respectively.
  - Absolute value (by implementing the `__abs__(self)` method)
  - A method `topolar` converting the complex number to a tuple `(r, theta)`. Hint, look at `math.atan2`.
  - A static method `frompolar(r, theta)` that returns an instance of `Complex`.
  - If `<` or `>` is used, an `UnorderedException` should be thrown, telling the user that complex numbers are unordered (can not be compared with e.g. `<`).
  - `==` between `Complex` numbers
- A global variable `im` defined as $1i$.

Examples

An example code and the expected output is shown below:

Code

```
from complex import Complex, im

print("Printing")
print(Complex(2,3))
print(Complex(5))
print(Complex(2, -3))

z1 = Complex(2, 3)
```

```
print("\nImag / Real")
print(z1.imag())
print(z1.real())

z2 = Complex(4, -2)

print("\nArithmethic")
print(z1 + z2)
print(3 + z1)
print(z1 * z2)
print(5 * z1)
print(z1 * 5)
print(abs(z1))
print(z1 == z1)

print("\nPolar")
r, theta = z1.topolar()
print("r = ", r, "theta = ", theta, "rad")
print(Complex.frompolar(r, theta))

print("\nException")
print(z1 < z1)
```

Output

```
2 + 3im
5 + 0im
2 - 3im

Imag / Real
3
2

Arithmethic
6 + 1im
5 + 3im
14 + 8im
10 + 15im
10 + 15im
3.605551275463989
True

Polar
r =  3.605551275463989 theta =  0.982793723247329 rad
2.0 + 3.0im

Exception
Traceback (most recent call last):
    [...]
complex.UnorderedException: Complex numbers are unordered
```

Question 4: Transposing a matrix (7p total)

Matrices are a building block for a lot of code, and many mathematical expressions require the use of a transpose to express in a convenient manner.

While this can be achieved by duplicating the memory; copying the data while switching rows and columns, it can also be accomplished by letting the matrix object itself remember whether or not it is transposed, without any need for "copies"!

Part A (1p)

Make a base class `Matrix` that store the size in `rows` and `cols`. This class should have 2 abstract methods;

- `.transpose()` - The transposing method returns a matrix with transposed values.
- `[i,j]` - The indexing operator for getting values from the matrix, i.e. `__getitem__`.

The true power of this transpose technique is shown when doing matrix algebra, but requires to much code for an exam

Part B (2p)

Create the subclass `DenseMatrix` which takes the argument `values` and `transposed` (default `False`) as inputs to it's constructor. Transposing should return a new matrix, but without copying `values`.

```
>>> x = DenseMatrix([[1,2,3],[4,5,6],[7,8,9]])
>>> x_t = x.transpose()
>>> print(x[1,2], x_t[1,2])
6 8
```

Part C (1p)

Create a abstract subclass `SymmetricMatrix` where you can trivially overload `tranpose()`.

The purpose of this class is to serve as a basis for all types of symmetric matrices, like diagonal, banded, sparse, etc. Though you will only need to implement one such subclass in part D

Hint: What happens what you transpose a symmetric matrix?

Part D (3p)

Create a subclass `BandedMatrix` (short for `BandedSymmetricMatrix`) from `SymmetricMatrix` which represents a banded symmetric matrix.

Banded matrices have all their nonzero values are stored close to the diagonal. This means they can efficiently store very large matrices by skipping out the sections where there are only zeroes.

Example of banded matrix and it's internal representation:

```
 1  2  3  0  0  0  0          banded = BandedMatrix([[1,2,3],
 2  4  5  6  0  0  0                                 [4,5,6],
 3  5  7  8  9  0  0                                 [7,8,9],
 0  6  8 10 11 12  0   ===>                         [10,11,12],
 0  0  9 11 13 14 15                                 [13,14,15],
 0  0  0 12 14 16 17                                 [16,17],
 0  0  0  0 15 17 18                                 [18]])
```

Test code:

```
def debug_print_matrix(m):
    for i in range(m.rows):
        for j in range(m.cols):
            print('{:2}'.format(m[i, j]), end=' ')
        print()


dense = DenseMatrix([[1,2,3,4,5],[5,4,3,2,1],[0,0,0,0,0],[1,1,1,1,1],[1,0,1,0,1]])

print('Dense matrix:')
debug_print_matrix(dense)
print('Dense matrix transpose:')
debug_print_matrix(dense.transpose())

banded = BandedMatrix([[1,2,3],[4,5,6],[7,8,9],[10,11],[12]])

print('Banded matrix:')
debug_print_matrix(banded)
print('Banded matrix transpose:')
debug_print_matrix(banded.transpose())
```

Output:

```
Dense matrix:
 1  2  3  4  5
 5  4  3  2  1
 0  0  0  0  0
 1  1  1  1  1
 1  0  1  0  1
Dense matrix transpose:
 1  5  0  1  1
 2  4  0  1  0
 3  3  0  1  1
 4  2  0  1  0
 5  1  0  1  1
Banded matrix:
 1  2  3  0  0
 2  4  5  6  0
 3  5  7  8  9
 0  6  8 10 11
 0  0  9 11 12
Banded matrix transpose:
 1  2  3  0  0
 2  4  5  6  0
 3  5  7  8  9
 0  6  8 10 11
 0  0  9 11 12
```